

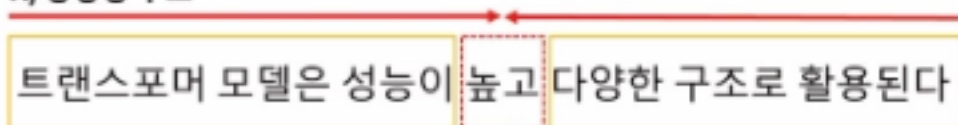
트랜스포머(Transformer)

Transformer: 2017년 코넬 대학의 “Attention is All You Need” 논문을 통해 소개된 신경망 아키텍처

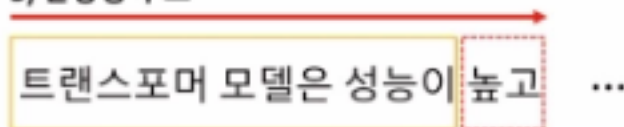
- 기존의 순환 신경망처럼 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리하는 기능 -> 긴 시퀀스를 훨씬 더 빠르고 효율적으로 처리할 수 있음.
- 기존 순환 신경망은 순차처리(sequential processing), 반복 연결(Recurrent connections)에 의존 하는것과 다르게 **어텐션 메커니즘(Attention Mechanism)**을 이용해 시퀀스 임베딩 표현
- **셀프 어텐션(Self attention)기반: 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있음**
-> 모델은 재귀나 합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링 할 수 있다.
- 대용량 데이터셋이 효율적(데이터의 양이 많은 기계 번역과 같은 작업에 적합)
- 언어 모델링, 텍스트 분류 같은 광범위한 자연어 처리 작업에 효율적(기계 번역, 언어 모델링, 텍스트 요약 등.. 장기적인 종속성을 포함하는 작업)

오토 인코딩(Auto-Encoding) : 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업(Task 수행). 예측되는 코트의 양옆에 있는 토큰 참조하는 양방향 구조(인코더)

A) 양방향 구조



B) 단방향 구조



단방향 구조는 예측되는 토큰의 왼쪽에 있는 토큰들만 참조: 디코더 (Decoder)

그림 7.1 트랜스포머 구조

자기 회귀(Auto Regressive) : 단방향성 디코더

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

트랜스포머의 어텐션 메커니즘: 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력 생성

-인코더: 입력 시퀀스 임베딩-> 고차원 벡터로 변환

-디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스 생성

어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중

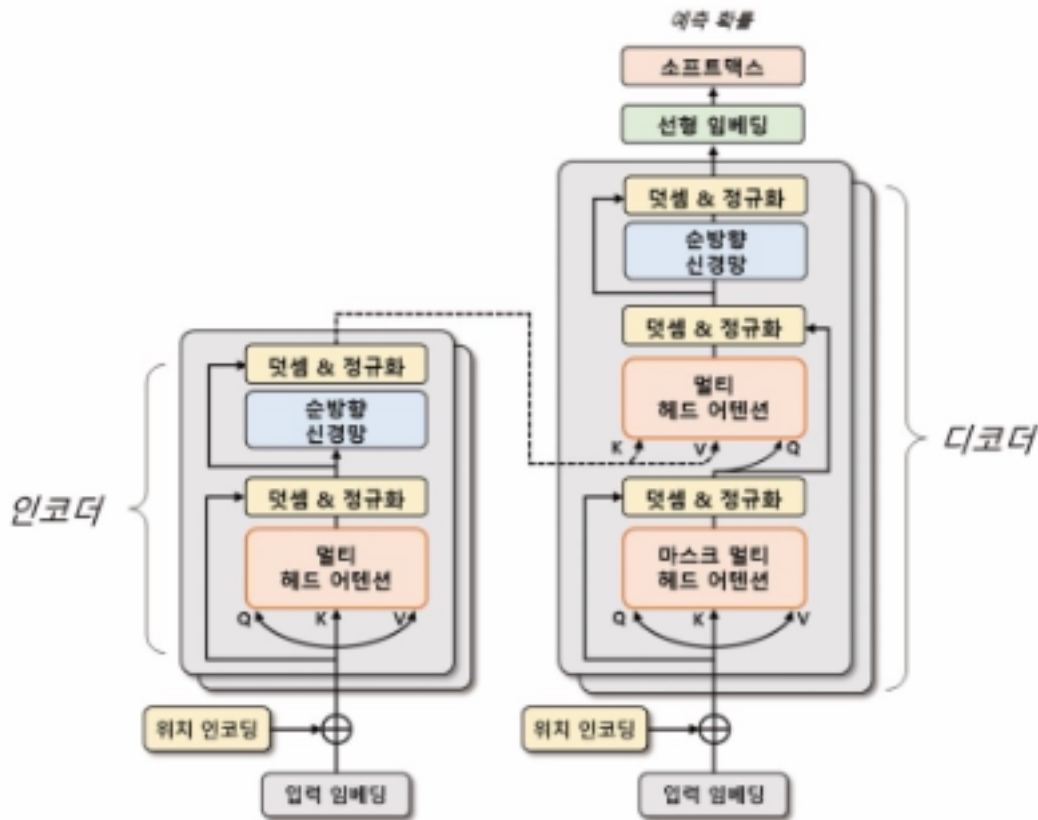


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

트랜스포머의 인코더와 디코더는 2 부분으로 구성, 각각 N개의 트랜스포머 블록(Transformer Block)으로 구성
 트랜스포머 블록은 멀티 헤드 어텐션, 순방향 신경망으로 구성

Multi-Head Attention: 입력 시퀀스에서 쿼리, 키, 값을 정의해 입력 시퀀스들의 관계를 셀프 어텐션으로 하는 벡터 표현 방법. 쿼리와 각 키의 유사도 계산 -> 해당 유사도를 가중치로 사용 -> 값 벡터 합산

이렇게 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터 대체 -> 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터 갱신. 임베딩 벡터 고도화시 순방향 신경망 사용.

순방향 신경망: 여러개의 선형 계층으로 구성. 입력 벡터에 가중치 곱, 편향 더해서 활성화 함수 적용.

트랜스포머에서는 입력 시퀀스 데이터를 소스(Source), 타겟(Target) 데이터로 나눠서 처리.

ex) 영 -> 한 번역 타겟 데이터: 한글, 소스 데이터: 영어

위치 인코딩(Positional Encoding): 인코더에서 소스 시퀀스를 입력 임베딩으로 표현하기 위해 데이터를 인코딩 하는 방법. 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성된다.

마스크 멀티 헤드 어텐션(Masked Multi-Head Attention): 디코더에서 타겟 시퀀스 데이터를 생성하기 위해 사용하는 방법.

생성된 디코더 출력 벡터는 선형 임베딩으로 재표현 -> 이미지나 자연어 모델에 활용.

트랜스포머는 챗봇, 기계 번역, 음성 인식 등 다양한 자연어 처리 작업에서 좋은 성능을 보임

입력 임베딩과 위치 인코딩

입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환.

병렬 구조로 입력 시퀀스 처리 -> 단어 순서 정보 제공x. 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 함 => 위치 인코딩 방식 사용

위치 인코딩 방식: 입력 시퀀스의 순서 정보를 모델에 전달하는 방법. 각 단어의 위치 정보를 나타내는 벡터를 더해 임베딩 벡터에 위치 정보 반영. sin함수와 cos함수를 사용해 위치 인코딩 벡터 생성

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

Pos: 입력 시퀀스에서 해당 단어의 위치

i: 임베딩 벡터의 차원 인덱스 -> 차원 인덱스 짝수: sin함수 수식 사용
-> 차원 인덱스 홀수: cos 함수 수식 사용

pe변수의 텐서 차원 [최대 시퀀스, 1, 입력 임베딩의 차원]

특수 토큰

트랜스포머는 단어 토큰 이외의 **특수 토큰**을 활용하여 문장 표현.

특수 토큰은 입력 시퀀스의 시작과 끝을 인식할 수 있게 하거나 마스킹을 통해 일부 입력을 무시할 수 있음.

ex)번역 모델: 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조.



특수 토큰: BOS(Beginning of Sentence): 문장의 시작 의미,

EOS(End of Sentence): 문장의 종료,

UNK(Unknown): 어휘 사전에 없는 단어/모르는 단어 처리할때 사용,

PAD(Padding): 모든 문장을 일정한 길이로 맞추기 위해 짧은 문장의 빈 공간 채울 때 사용

자연어 처리에서 일반적으로 사용

생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현

ex) '트랜스포머'라는 원-핫 벡터가 입력 임베딩으로 변환되는 과정(Word2Vec과 동일)

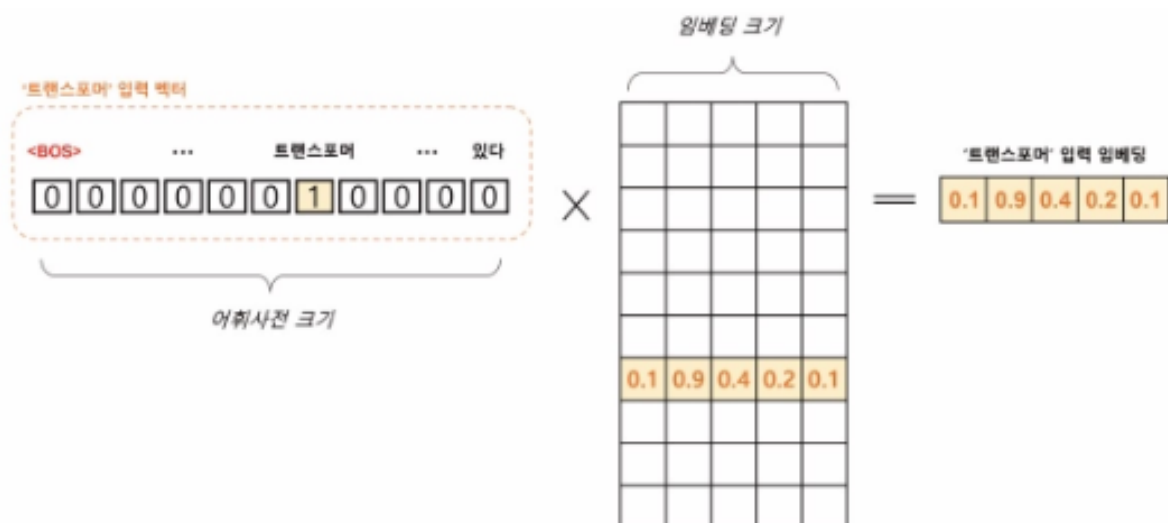


그림 7.4 입력 임베딩 산출 과정

트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행
- 멀티 헤드 어텐션, 순방향 신경망으로 구성, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달
- 인코더 계층에서 위치 임베딩 벡터를 입력 벡터에 더함-> 인코더 계층의 출력을 디코더 계층으로 전달

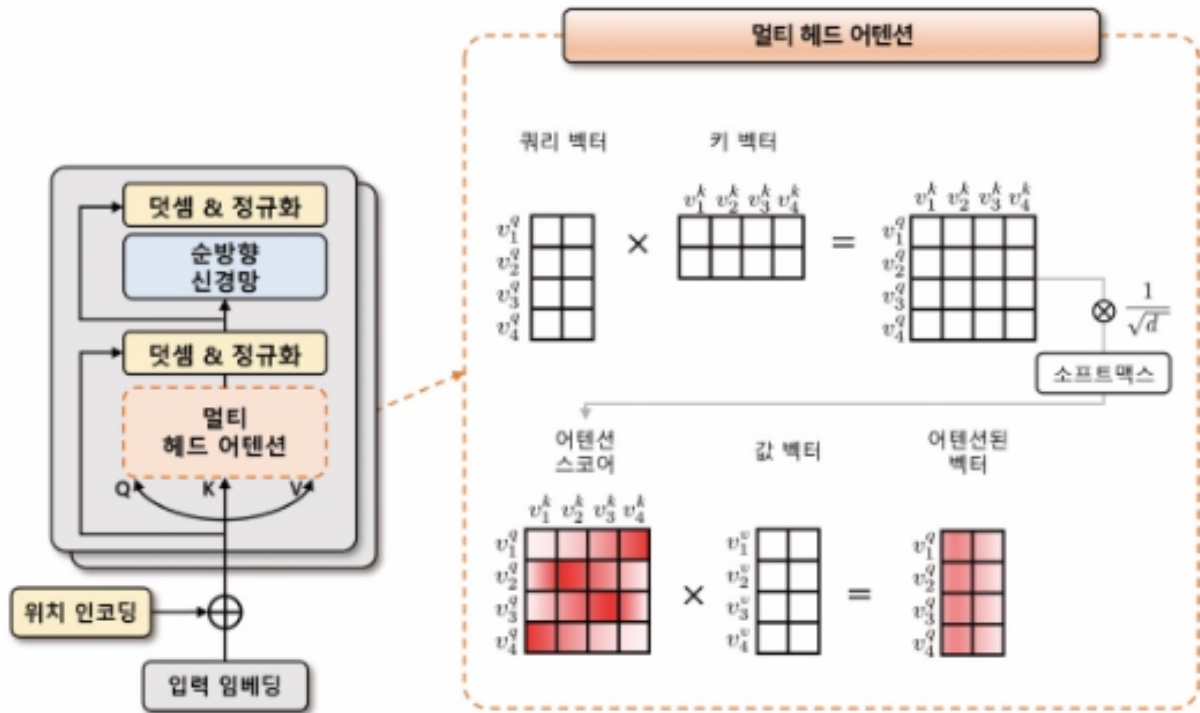


그림 7.5 트랜스포머 인코더 블록 연산 과정

멀티 헤드 어텐션 단계에서는 입력 데이터 차원이 $[N, S, d]$ 라고 한다면 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터 생성 -> 3개의 벡터는 각각 쿼리(Q), 키(K), 값(V)라고 정의

쿼리 벡터: 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터(질문이 되는 벡터). 인코더의 각 시점마다 생성. 해당 벡터를 기준으로 다른 시점의 정보 참조

키 벡터: 쿼리 벡터와 비교되는 대상. 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터. 인코더의 각 시점에서 생성

값 벡터: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

=> 쿼리, 키, 값 벡터는 초기에는 비슷한 값을 가지지만 모델 학습을 통해 각 벡터는 의도한 의미의 값을 갖게 됨.

내적 연산으로 $[N, S, S]$

어텐션 스코어 맵을 사용

$$\text{score}(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

보정된 어텐션 스코어를 소프트맥스 함수를 이용-> 확률적으로 재표현-> 값 벡터와 내적-> 셀프 어텐션된 벡터 생성->반복...-> 셀프 어텐션된 스코어 맵 생성

멀티 헤드: 셀프 어텐션을 여러 번 수행해 여러 개의 헤드 생성 -> 각각의 세트가 독립적으로 어텐션을 수행하고 그 결과를 합침.

입력받은 $[N, S, d]$ 텐서에 k개의 셀프 어텐션 벡터를 생성하려하면 헤드에 대한 차원축을 생성해 $[N, k, S, d/k]$ 텐서 형태를 구성. 이 텐서는 k개의 셀프 어텐션된 $[N, S, d/k]$ 텐서를 의미.

이렇게 생성된 k개의 셀프 어텐션 벡터는 임베딩 차원축으로 다시 병합(concatenation)되어 $[N, S, d]$ 의 형태로 출력

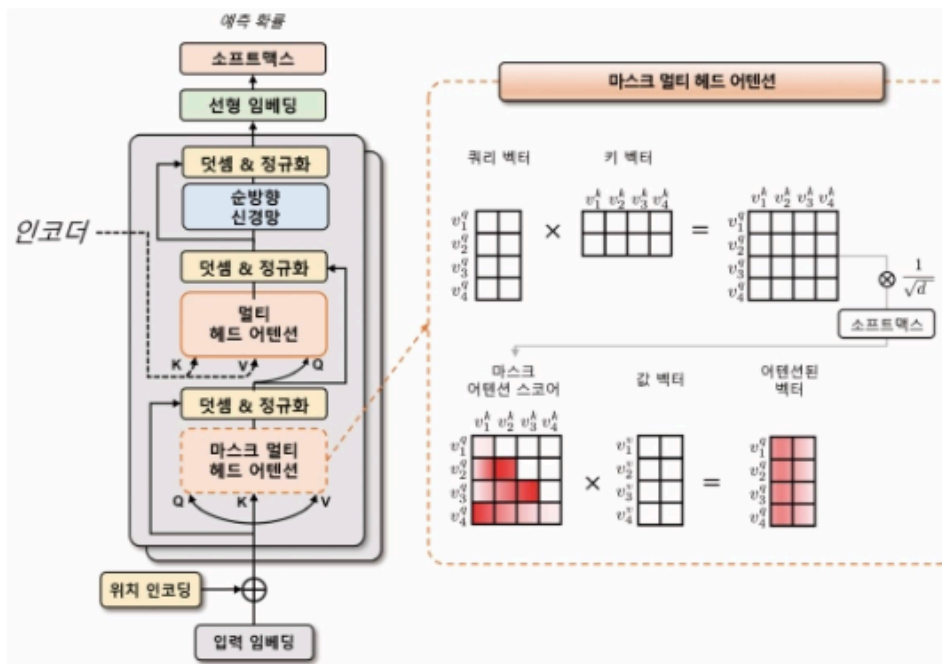
트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성. 이전 블록에서 출력된 벡터-> 다음 블록의 입력으로 전달되어 인코더 블록을 통과-> 입력 시퀀스 정보 추상화.
마지막 인코더 블록에서 출력된 벡터-> 디코더에서 사용. 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용

트랜스포머 디코더

위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력 받아 입력 임베딩에 위치 정보를 추가함으로써 디코더가 입력 시퀀스의 순서 정보를 학습할 수 있게 함.

트랜스포머 모델에서 인코더의 멀티 헤드 모듈은 인과성(Casuality)을 반영한 **마스크 멀티 헤드 어텐션** 모듈로 대체

마스크 멀티 헤드 어텐션 모듈은 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크를 씌움, 두 번째 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움 -> 셀프 어텐션에서 현재 위치 이전의 단어들만 참조->인과성 보장



타깃 데이터: 쿼리 벡터로 사용, 인코더의 소스 데이터가 키와 값 벡터로 사용 -> 쿼리 벡터는 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 됨

쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵 계산->소프트맥스 함수를 적용해 어텐션 가중치 구함 -> 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻음

마지막 디코더 블록의 출력 텐서[N, S, d]에 선형 변환 및 소프트맥스 함수를 적용해 각 타식 시퀀스 위치마다 예측 확률 계산 가능.

PAD토큰으로 입력 토큰 순차적으로 나타낼 수 있음

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]	[BOS] 'ChatGPT', [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
2	[BOS], 'ChatGPT', [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]	[BOS], 'ChatGPT', '는', [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
3	[BOS], 'ChatGPT', '는', [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD] [PAD] [PAD] [PAD] [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD] [PAD] [PAD] [PAD] [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD] [PAD] [PAD] [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD] [PAD] [PAD] [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD] [PAD] [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄 져', [PAD] [PAD] [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있 다', [PAD] [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄 져', '있다', [PAD] [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있 다', [EOS] [PAD]

GPT

GPT(Generative Pre-trained Transformer): 2018 openAI에서 발표한 트랜스포머 기반 언어 모델.

트랜스포머의 디코더(입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점)를 여러 층 쌓아 만든 언어 모델

ELMo(Embedding from Language Model과 같이 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능을 보임.

GPT1

트랜스포머 구조를 바탕으로 한 단방향 언어 모델.

이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작-> 계산량 감소로 빠른 학습속도와 작은 모델 크기의 장점

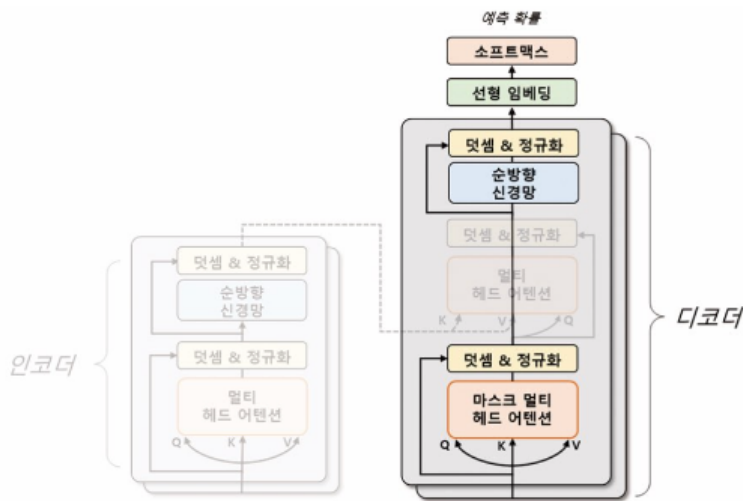


그림 7.7 트랜스포머와 GPT-1 모델 비교

디코더만 사용함 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조 활용->모델의 매개변수 수를 줄이면서 높은 성능.

레이블 없는 비지도 학습->컴퓨팅 자원만 충분하다면 제약 없이 학습 가능

GPT-1의 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산

미세 조정을 위한 다운 스트림 작업에서도 GPT-1은 각 작업에 따라 입출력 구조를 바꾸지 않고 언어모델을 보조 학습(Auxiliary Learning: 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행) 해서 사용

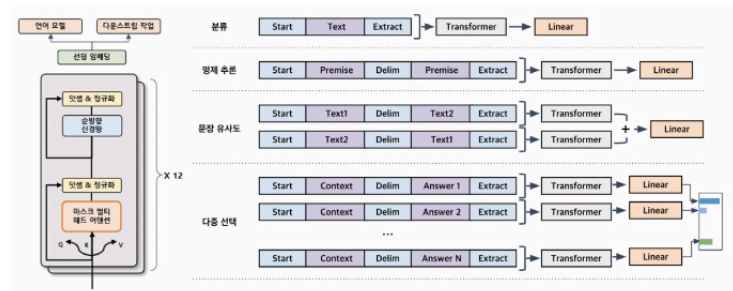
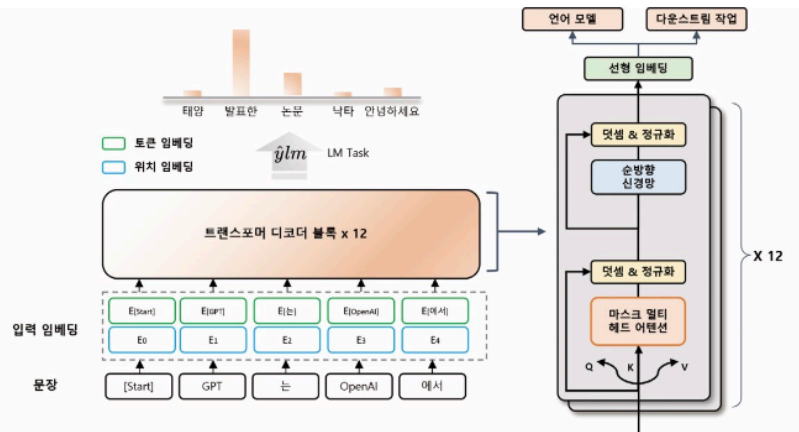


그림 7.9 GPT-1의 다운스트림 작업 구조

GPT2

1에 비해 디코더 갯수: 12->48. 매개변수 갯수: 1,200만 -> 15억 => 더 복잡한 패턴 학습 가능

더 자연스러운 문장 생성 능력

제로-샷 학습 가능: 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력

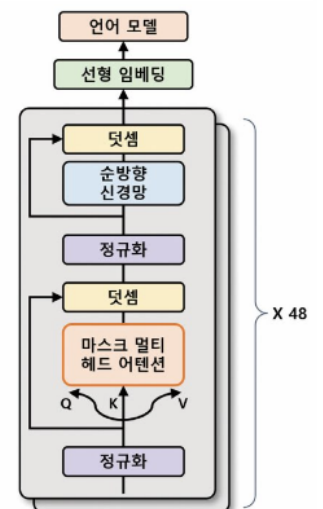


그림 7.10 GPT-2의 모델 구조

GPT3

gpt2보다 더 많은 매개 변수

멀티 헤드 어텐션 사용. 이때 연산량을 줄이기 위해 희소 어텐션과 일반 어텐션을 섞어서 사용

gpt2처럼 프롬프트 형식으로 작업

느리고 비용이 많이 든다는 문제점. 사전 학습된 데이터에 기반하여 작동-> 편향된 데이터 문제

GPT3.5

InstructGPT라고도 불림. gpt3의 구조를 따르면서 동시에 새로운 학습 방법인 RLHF를 도입해 모델의 매개변수 수를 줄이고 모델의 자연스러움을 높인 특징.

RLHF: 데이터셋에서 임의의 프롬프트 가져오 사람이 적절한 답 작성 -> 작성된 답변은 모델이 생성한 문장과 함께 평가 -> if 모델 답변이 자연스러움면 보상->강화학습 : Supervised Fine-Tuning

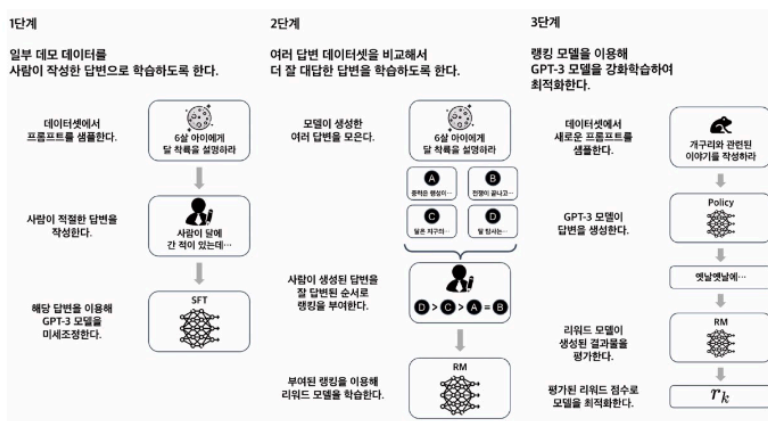


그림 7.12 RLHF GPT-3 학습 방법

사람이 전체 프롬프트에 답변다는건 어려움 -> 보상 모델을 학습

GPT4

텍스트 데이터뿐 아니라 이미지 데이터도 인식 가능한 멀티모달 모델

높은 성능 향상을 보이지만 환각 현상은 아직 해결x